Distributed Performance Testing Platform

Project Proposal for UCS503P
Submitted to: Dr. Paramveer Kaur
Thapar Institute of Engineering and Technology

Jatin Bajaj - 1024170389
Krish Sahni - 1024170390
Rishabh Dutt - 10240170393

August 16, 2026

1 Higher-order goal

. . . primary objective

This project aims to build a scalable, backend-focused distributed performance testing platform capable of generating and analyzing high volumes of concurrent HTTP traffic against APIs and web applications.it focuses primarily on distributed backend architecture, scalable request generation, real-time telemetry processing, and automated performance analysis.

The objective is not merely to generate load and display graphs, but to provide an engineering platform capable of determining system capacity, identifying performance bottlenecks, detecting breaking points, and comparing performance across different application versions.

2 Time-to-value

. . . primary heuristic

- The initial iteration will focus on implementing the core controller-worker architecture and generating concurrent HTTP traffic against configurable API endpoints.
- Subsequent iterations will introduce distributed workers, telemetry streaming, metric aggregation, real-time dashboards, adaptive stress testing, and performance regression testing.
- Development will prioritize measurable backend functionality over visual complexity, with each iteration producing a testable performance-testing capability.

3 Problem Statement

Existing performance-testing tools can generate large volumes of requests, but configuring, distributing, analyzing, and continuously comparing performance results can become complex for backend developers.

The proposed platform addresses the following challenges:

- **Limited scalability:** A single load generator may become a bottleneck when thousands of concurrent requests are required.
- **Distributed load generation:** Large workloads need to be distributed across multiple machines or worker processes.

- **Limited real-time analysis:** Raw request results need to be aggregated into useful metrics such as throughput, latency percentiles, and error rates.
- **Manual breaking-point detection:** Developers often need to manually increase traffic and inspect results to determine when a system becomes unstable.
- **Performance regressions:** Performance degradation between application versions may not be immediately visible without systematic comparison of historical test results.

4 Proposed Solution

... software proposition

4.1 Overview

Build a backend-focused distributed load-testing platform consisting of a central controller, multiple load-generator workers, a metrics aggregation pipeline, persistent test storage, and a real-time monitoring interface.

The platform will provide:

- **Central controller:** Creates, configures, schedules, starts, and stops performance tests.
- **Load-generator workers:** Generate HTTP traffic concurrently and execute assigned workloads.
- **Metrics aggregation pipeline:** Processes high-volume telemetry generated by workers and calculates performance metrics.
- **Historical storage:** Stores test configurations, execution results, aggregated metrics, and performance history.
- **Real-time dashboard:** Streams test progress and performance metrics to users using WebSockets.

4.2 Core workflow

1. The user creates a performance test by configuring the target endpoint, virtual users, request rate, ramp-up period, duration, authentication, headers, payloads, and scenarios.
2. The central controller validates the configuration and distributes the workload across available load-generator workers.
3. Workers generate concurrent HTTP requests according to the assigned workload and collect request-level telemetry.
4. Workers asynchronously stream telemetry to the metrics aggregation pipeline.
5. The aggregation layer calculates throughput, requests per second, latency percentiles, error rate, timeouts, response codes, and concurrent requests.
6. Aggregated metrics are persisted and simultaneously streamed to the dashboard for real-time monitoring.
7. After execution, the platform evaluates configured SLOs and determines whether the application remained within acceptable performance limits.

5 Performance Metrics *... measurable output*

The platform will calculate and expose the following metrics:

- **Throughput:** Total number of successfully processed requests during a test.
- **Requests per second (RPS):** Number of requests generated or completed per second.
- **Latency percentiles:** P50, P95, and P99 latency values representing typical, high, and worst-case response behavior.
- **Error rate:** Percentage of requests resulting in failures.
- **Timeouts:** Number and percentage of requests that exceeded the configured timeout.
- **Response codes:** Distribution of HTTP status codes such as 2xx, 4xx, and 5xx responses.
- **Concurrency:** Number of active requests or virtual users during different stages of the test.

6 Adaptive Stress Testing *... capacity discovery*

Instead of using only a fixed load, the platform will support adaptive stress testing.

The system will progressively increase traffic according to a configured strategy and continuously evaluate application performance against predefined Service Level Objectives (SLOs).

Example SLOs include:

- P95 latency must remain below a configured threshold.
- P99 latency must remain below a configured threshold.
- Error rate must remain below a configured percentage.
- Request timeout rate must remain below a configured limit.

The **maximum sustainable throughput** will be identified as the highest load at which the application continues to satisfy the configured SLOs.

The **breaking point** will be identified when increasing load causes the application to violate one or more critical SLOs, such as a significant increase in latency, error rate, or timeouts.

This allows the system to automatically determine the approximate capacity boundary of the tested application rather than requiring manual inspection.

7 Performance Regression Testing *... version comparison*

The platform will support performance regression testing by storing historical test results and associating them with application versions or builds.

A new performance test can be compared against a previous baseline to identify degradation in:

- P50/P95/P99 latency.
- Throughput and requests per second.
- Error and timeout rates.
- Maximum sustainable throughput.

- Resource or concurrency behavior when such measurements are available.

A performance regression occurs when a newer application version performs significantly worse than the established baseline under comparable test conditions.

This functionality also makes the platform suitable for integration into a CI/CD pipeline, where performance tests can be executed automatically for application builds.

8 System Architecture

... backend engineering focus

The proposed platform will follow a distributed architecture:

- **Controller:** Maintains test configuration, manages test lifecycle, schedules workloads, and coordinates workers.
- **Load Generators:** Horizontally scalable workers that generate concurrent HTTP traffic.
- **Telemetry Pipeline:** Receives high-volume execution data asynchronously from workers.
- **Metrics Aggregator:** Converts raw telemetry into aggregated performance metrics and percentile statistics.
- **Persistent Storage:** Stores test definitions, execution metadata, historical metrics, and regression baselines.
- **WebSocket Layer:** Provides real-time performance updates to connected dashboard clients.

The architecture is designed so that additional load-generator workers can be added horizontally when higher traffic generation capacity is required.

9 Solution Approach

... engineering focus

The project will focus on backend engineering challenges rather than AI/ML-based prediction.

9.1 Concurrent Request Generation

Workers will use asynchronous or non-blocking networking techniques to efficiently generate a large number of concurrent HTTP requests without requiring one dedicated blocking thread for every request.

9.2 Distributed Coordination

The controller will divide workload among workers and maintain the state of active performance tests. Workers will report their execution status and telemetry back to the controller or telemetry pipeline.

9.3 Rate Limiting

Request generation will support configurable request rates to prevent uncontrolled traffic generation and to implement controlled load profiles.

9.4 Telemetry Streaming

Workers will stream high-volume execution data rather than waiting until the complete test finishes. This enables real-time aggregation and monitoring.

9.5 Fault Tolerance

The platform will account for worker failures and communication issues so that individual worker failures do not unnecessarily terminate the complete testing infrastructure.

10 Real-time Monitoring *... live system visibility*

During a running test, aggregated metrics will be transmitted to the dashboard through WebSockets.

The dashboard will provide real-time visibility into:

- Current requests per second.
- Active concurrent requests.
- P50, P95, and P99 latency.
- Error and timeout rates.
- HTTP response-code distribution.
- Current load level during adaptive stress testing.
- SLO violations and detected performance limits.

This avoids the need to wait until the test finishes before understanding the application's current behavior.

11 CI/CD Integration *... automated regression detection*

The platform can be integrated into CI/CD pipelines so that performance tests become an automated quality gate.

A typical pipeline can execute the following workflow:

1. Build a new application version.
2. Deploy the application to a test environment.
3. Trigger the performance-testing platform through an API.
4. Execute a predefined workload.
5. Collect and aggregate performance metrics.
6. Compare results against the stored baseline.
7. Mark the pipeline as successful or failed according to configured performance SLOs.

This enables performance degradation to be detected before a build is released to production.

12 Evaluation Criteria *... measurable and attributable*

The project will be evaluated using measurable technical metrics.

12.1 Primary Evaluation Metrics

- **Load generation capacity:** Maximum requests per second generated by the platform.
- **Concurrency:** Maximum number of concurrent virtual users and requests supported.
- **Telemetry throughput:** Number of execution events processed by the aggregation pipeline per second.
- **Metric accuracy:** Correctness of calculated throughput, latency percentiles, error rates, and response-code distributions.

12.2 Secondary Evaluation Metrics

- Controller-to-worker coordination latency.
- Worker failure recovery behavior.
- Real-time dashboard update latency.
- Storage scalability for historical test results.
- Accuracy of maximum sustainable throughput and breaking-point detection.

13 Scalability *. . . horizontal growth*

The system is designed for horizontal scalability.

1. **Load-generator scalability:** Additional workers can be added to increase traffic-generation capacity.
2. **Aggregation scalability:** Telemetry processing can be distributed across multiple aggregation instances when event volume increases.
3. **Stateless services:** Controller and API services can be designed to remain stateless where possible, allowing multiple instances to run concurrently.
4. **Storage scalability:** Historical performance data can be indexed and partitioned according to test, timestamp, and application version.
5. **Asynchronous processing:** Streaming and asynchronous processing prevents high-volume telemetry from blocking request generation.

14 Project Scope and Deliverables *. . . iteration-friendly*

14.1 Initial Deliverable *. . . first iteration*

- Central controller for creating and managing tests.
- Single load-generator worker capable of generating configurable concurrent HTTP requests.
- Configurable virtual users, request rate, duration, ramp-up/ramp-down, headers, authentication, and payloads.
- Collection of request-level performance metrics.
- Basic metrics aggregation for throughput, latency, response codes, and errors.
- Persistent storage of test configurations and results.

14.2 Subsequent Deliverables

- Multiple distributed load-generator workers.
- Streaming telemetry pipeline.
- Real-time WebSocket dashboard.
- P50/P95/P99 latency analysis.
- Adaptive stress testing and breaking-point detection.
- Configurable SLO-based performance evaluation.
- Historical performance comparison.
- Performance regression testing.
- CI/CD pipeline integration.
- Worker failure handling and fault tolerance.

15 Risks and Mitigations

- **Load-generator bottleneck:** Use asynchronous networking and distribute traffic across multiple workers.
- **Telemetry overload:** Use batching, asynchronous streaming, and aggregation instead of storing every event synchronously.
- **Incorrect percentile calculations:** Use validated percentile algorithms and test metric calculations against known datasets.
- **Worker failure:** Track worker heartbeats and isolate failed workers from the remaining test execution.
- **False breaking-point detection:** Evaluate multiple metrics and require configurable SLO violations rather than relying on a single latency measurement.
- **Storage growth:** Store aggregated metrics for long-term analysis while limiting unnecessary raw telemetry retention.

16 Summary

This project proposes a backend-focused distributed performance testing platform capable of generating large-scale HTTP workloads using a central controller and horizontally scalable load-generator workers. Unlike a basic load-testing tool that only generates requests and plots graphs, the proposed system focuses on distributed workload orchestration, asynchronous networking, telemetry streaming, real-time metric aggregation, performance visualization, adaptive stress testing, SLO-based capacity analysis, breaking-point detection, and performance regression testing. The visualization layer will transform aggregated telemetry into real-time performance graphs, latency percentile trends, throughput and concurrency charts, response-code distributions, SLO indicators, capacity visualizations, and historical comparisons. The platform will provide a practical environment for studying and implementing backend engineering concepts including high concurrency, distributed systems, rate limiting, asynchronous processing, fault tolerance, scheduling, real-time communication, performance benchmarking, CI/CD quality gates, performance visualization, and scalable data storage.